

Load the data. You'll need to put the data file in your current working directory, or equivalent to this `load()` function, you can double-click it. Use the `str()` function to take a peek at the types of variables in the data.

The survey (and it's results) are described here:

<https://fivethirtyeight.com/features/americas-favorite-star-wars-movies-and-least-favorite-characters/>

```
> load('StarWarsSurvey.RData')
> str(starwars)
```

```
'data.frame':      1186 obs. of  38 variables:
 $ RespondentID      : num  3.29e+09 3.29e+09 3.29e+09 3.29e+09 3.29e+09 ...
 $ SeenAnyMovie      : Factor w/ 3 levels "No","Yes",NA: 2 1 2 2 2 2 2 2 2 ...
 $ AreYouAFan        : Factor w/ 3 levels "No","Yes",NA: 2 3 1 2 2 2 2 2 1 ...
 $ SeenEpisodeI      : num  1 0 1 1 1 1 1 1 1 0 ...
 $ SeenEpisodeII     : num  1 0 1 1 1 1 1 1 1 1 ...
 $ SeenEpisodeIII    : num  1 0 1 1 1 1 1 1 1 0 ...
 $ SeenEpisodeIV     : num  1 0 0 1 1 1 1 1 1 0 ...
 $ SeenEpisodeV      : num  1 0 0 1 1 1 1 1 1 0 ...
 $ SeenEpisodeVI     : num  1 0 0 1 1 1 1 1 1 0 ...
 $ RankEpisodeI      : num  3 0 1 5 5 1 6 4 5 1 ...
 $ RankEpisodeII     : num  2 0 2 6 4 4 5 5 4 2 ...
 $ RankEpisodeIII    : num  1 0 3 1 6 3 4 6 6 3 ...
 $ RankEpisodeIV     : num  4 0 4 2 2 6 3 3 2 4 ...
 $ RankEpisodeV      : num  5 0 5 4 1 5 1 2 1 5 ...
 $ RankEpisodeVI     : num  6 0 6 3 3 2 2 1 3 6 ...
 $ ViewHanSolo       : num  5 0 4 5 5 5 5 5 5 3 ...
 $ ViewLukeSkywalker : num  5 0 4 5 4 5 5 4 2 5 ...
 $ ViewPrincessLeiaOrgana : num  5 0 4 5 4 5 4 5 4 5 ...
 $ ViewAnakinSkywalker : num  5 0 4 5 2 5 4 3 4 5 ...
 $ ViewObiWanKenobi  : num  5 0 4 5 5 5 5 5 4 5 ...
 $ ViewEmperorPalpatine : num  5 0 0 4 1 3 5 1 5 2 ...
 $ ViewDarthVader    : num  5 0 0 5 4 5 5 2 5 5 ...
 $ ViewLandoCalrissian : num  0 0 0 4 3 3 5 3 5 2 ...
 $ ViewBobaFett      : num  0 0 0 2 5 4 5 4 5 2 ...
 $ ViewC3P0          : num  5 0 0 5 4 4 4 4 3 5 ...
 $ ViewR2D2          : num  5 0 0 5 4 4 5 4 4 5 ...
 $ ViewJarJarBinks   : num  5 0 0 5 1 4 2 1 1 5 ...
 $ ViewPadmeAmidala  : num  5 0 0 5 4 3 4 2 2 2 ...
 $ ViewYoda          : num  5 0 0 5 4 5 5 5 4 5 ...
 $ WhichCharacterShotFirst : Factor w/ 4 levels "Greedo","Han",...: 3 4 3 3 1 2 2 2 2 3 ...
 $ FamiliarWithExpandedUniverse : Factor w/ 3 levels "No","Yes",NA: 2 3 1 1 2 2 2 1 1 1 ...
 $ FanOfExpandedUniverse : Factor w/ 3 levels "No","Yes",NA: 1 3 3 3 1 1 1 3 3 3 ...
 $ FanOfStarTrek      : Factor w/ 3 levels "No","Yes",NA: 1 2 1 2 1 2 1 2 1 1 ...
 $ Gender             : Factor w/ 3 levels "Female","Male",...: 2 2 2 2 2 2 2 2 2 2 ...
 $ Age                : Factor w/ 5 levels "> 60","18-29",...: 2 2 2 2 2 2 2 2 2 2 ...
 $ HouseholdIncome     : Factor w/ 6 levels "$0 - $24,999",...: 6 1 1 2 2 4 6 6 1 4 ...
 $ Education           : Factor w/ 6 levels "Bachelor degree",...: 3 1 3 5 5 1 3 3 5 5 ...
 $ Location            : Factor w/ 10 levels "East North Central",...: 7 9 8 8 8 3 1 7 7 6 ...
```

There are lots of factor variables which are not suited for PCA input (PCA functions will undoubtedly want you to input numeric data, regardless of software platform). While it's not the *most* principled thing to use dummy variables as input to PCA (we prefer to think about PCA in terms of elliptically distributed numeric data), it's common and often works. The math holds just the same - we still get a projection of our data that is of maximal variance. We're going to proceed with that approach here - though you are welcome to further exploration without the binary and factor columns.

To create the dummy variables for all factors, we make a model matrix with **one-hot encoded** factors (this means we don't leave out a reference column). The `contrasts.arg=` option in the following `model.matrix()` function will do the one-hot encoding.

```
> starwars.x=model.matrix(RespondentID~. ,contrasts.arg = lapply(starwars[,sapply(starwars, is.factor) ], contrasts, contr
```

Print the dimensions of the final matrix (with dummy variables enumerated):

```
> dim(starwars.x)
```

```
[1] 1186    76
```

This is the *true dimensionality* of your data! If you were asked "how many columns are in your dataset?" you would be wise to answer 76 rather than 38!

The `prcomp()` function is the one I most often recommend for reasonably sized principal component calculations in R. This function returns a list with class "prcomp" containing the following components (from help prcomp):

1. **sdev**: the standard deviations of the principal components (i.e., the square roots of the eigenvalues of the covariance/correlation matrix, though the calculation is actually done with the singular values of the data matrix).
2. **rotation**: the matrix of *variable loadings* (i.e., a matrix whose columns contain the eigenvectors). The function `princomp` returns this in the element `loadings`.
3. **x**: if `retx` is true *the value of the rotated data (i.e. the scores)* (the centred (and scaled if requested) data multiplied by the rotation matrix) is returned. Hence, `cov(x)` is the diagonal matrix $\text{diag}(\text{sdev}^2)$. For the formula method, `napredict()` is applied to handle the treatment of values omitted by the `na.action`.
4. **center, scale**: the centering and scaling used, or FALSE.

The option `scale = TRUE` inside the `prcomp()` function instructs the program to use **correlation PCA**. The **default is covariance PCA**.

```
> pca = prcomp(starwars.x, scale =T)
```

An error message! Cannot rescale a constant/zero column to unit variance. Solution: check for columns with zero variance and remove them. Recheck dimensions of the matrix to see how many columns we lost.

```
> starwars.x = starwars.x[,apply(starwars.x, 2, sd)>0 ]  
> dim(starwars.x)
```

```
[1] 1186    74
```

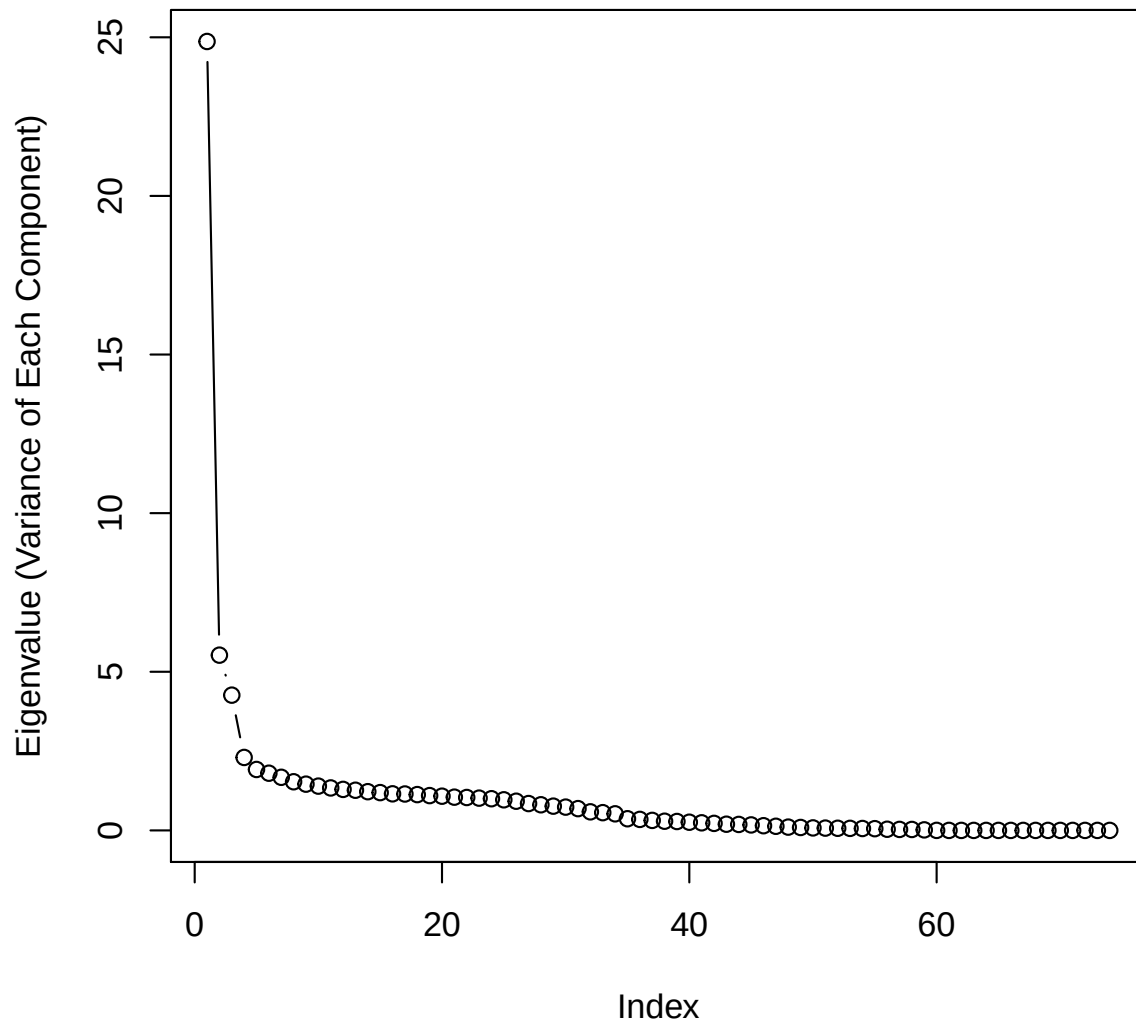
We can now compute the principal components of the matrix.

```
> pca = prcomp(starwars.x, scale =T)
```

To get the screeplot:

```
> plot(pca$sdev^2, type='b', ylab = 'Eigenvalue (Variance of Each Component)',  
+      main = 'Starwars Screeplot')
```

Starwars Screeplot

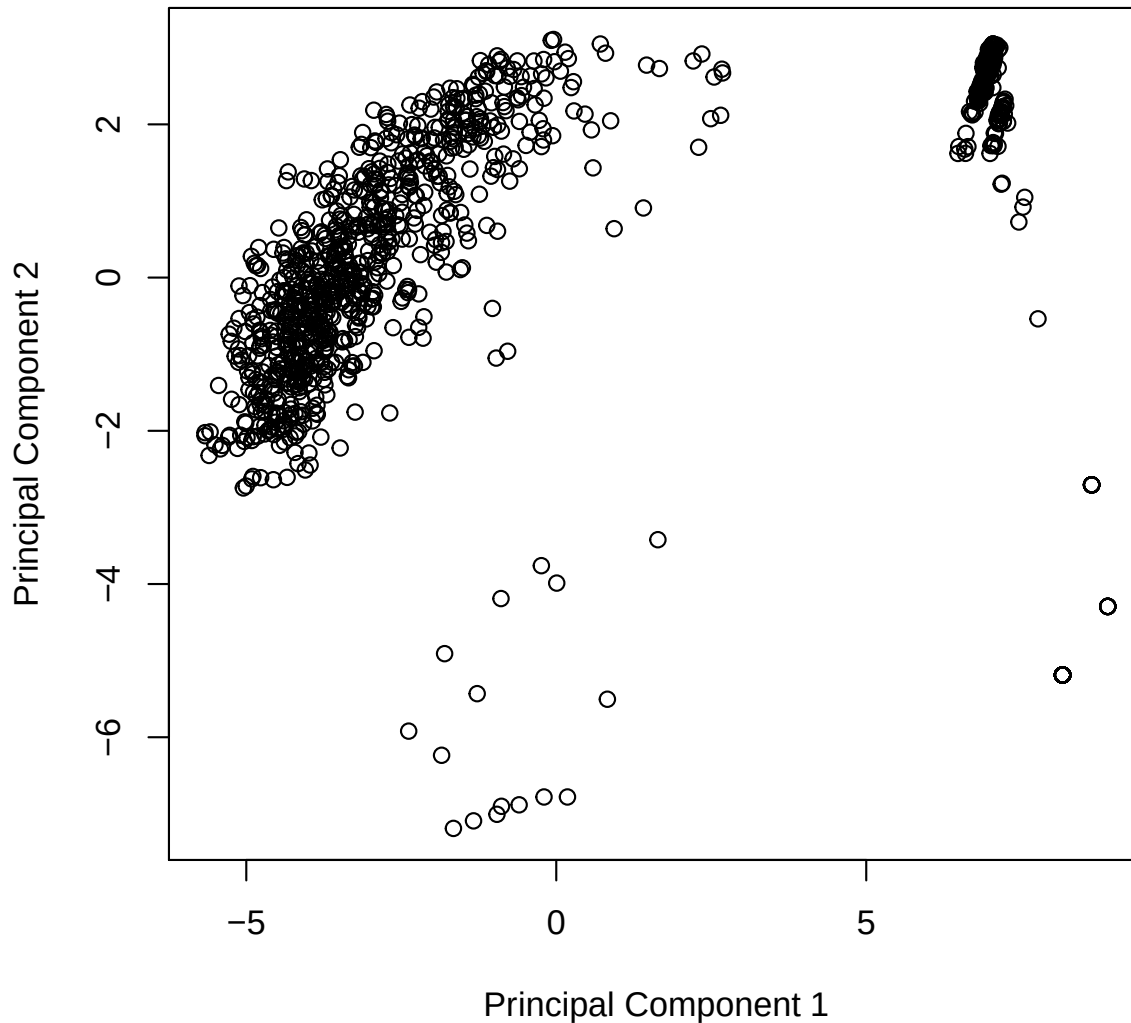


Projection of data in 2 dimensions

```
> plot(pca$x[,1:2], xlab = "Principal Component 1",  
+      ylab = "Principal Component 2",  
+      main='Projection of the 1186 Survey Respondents  
+          \n in 2-dimensional space')
```

Projection of the 1186 Survey Respondents

in 2-dimensional space



It is common for points to have the same or very similar scores along PCs, particularly when each variable has a limited range of values (i.e. binary or likert scale). This means that many points can be overlapping on a plot and that can be misleading. A common solution is to "jitter" the plot which merely adds some random error to the placement of the points so that you can see overlapping groups of points with more clarity:

```
> plot(jitter(pca$x[,1:2],amount=0.1), xlab = "Principal Component 1",
+       ylab = "Principal Component 2",
+       main='Projection of the 1186 Survey Respondents
+       \n in 2-dimensional space (w/Jitter)')
```

Note the clusters of individuals outstanding on PC1. Let's investigate the dominant variables driving PC1. We'll simply look at the loadings of variables on PC1, ordered by the absolute value of their magnitude.

```
> head(pca$rotation[order(abs(pca$rotation[,1]),decreasing=T),1],10)
```

AreYouAFanNA	WhichCharacterShotFirstNA
0.1936650	0.1920700
FamiliarWithExpandedUniverseNA	ViewHanSolo
0.1920700	-0.1908215
ViewLukeSkywalker	ViewPrincessLeiaOrgana
-0.1904492	-0.1901918
ViewR2D2	ViewYoda
-0.1889610	-0.1888073

ViewObiWanKenobi
-0.1873507

ViewC3PO
-0.1840787

The 3 largest loadings involve levels from missing values. In this case, I'd like to just take a quick look at the data of interest, those observations that have scores on PC1 that are greater than 5. So I'll create a temporary dataset to explore

```
> look = starwars[pca$x[,1]>5, ]  
> head(look)
```

	RespondentID	SeenAnyMovie	AreYouAFan	SeenEpisodeI	SeenEpisodeII	
2	3292879538	No	<NA>	0	0	
11	3292637870	Yes	<NA>	0	0	
12	3292635062	No	<NA>	0	0	
26	3292447974	No	<NA>	0	0	
35	3292297848	No	<NA>	0	0	
47	3292202220	No	<NA>	0	0	
	SeenEpisodeIII	SeenEpisodeIV	SeenEpisodeV	SeenEpisodeVI	RankEpisodeI	
2	0	0	0	0	0	
11	0	0	0	0	0	
12	0	0	0	0	0	
26	0	0	0	0	0	
35	0	0	0	0	0	
47	0	0	0	0	0	
	RankEpisodeII	RankEpisodeIII	RankEpisodeIV	RankEpisodeV	RankEpisodeVI	
2	0	0	0	0	0	
11	0	0	0	0	0	
12	0	0	0	0	0	
26	0	0	0	0	0	
35	0	0	0	0	0	
47	0	0	0	0	0	
	ViewHanSolo	ViewLukeSkywalker	ViewPrincessLeiaOrgana	ViewAnakinSkywalker		
2	0	0		0	0	
11	0	0		0	0	
12	0	0		0	0	
26	0	0		0	0	
35	0	0		0	0	
47	0	0		0	0	
	ViewObiWanKenobi	ViewEmperorPalpatine	ViewDarthVader	ViewLandoCalrissian		
2	0		0	0	0	
11	0		0	0	0	
12	0		0	0	0	
26	0		0	0	0	
35	0		0	0	0	
47	0		0	0	0	
	ViewBobaFett	ViewC3PO	ViewR2D2	ViewJarJarBinks	ViewPadmeAmidala	ViewYoda
2	0	0	0	0	0	0
11	0	0	0	0	0	0
12	0	0	0	0	0	0
26	0	0	0	0	0	0
35	0	0	0	0	0	0
47	0	0	0	0	0	0
	WhichCharacterShotFirst	FamiliarWithExpandedUniverse	FanOfExpandedUniverse			
2	<NA>		<NA>	<NA>		
11	<NA>		<NA>	<NA>		
12	<NA>		<NA>	<NA>		
26	<NA>		<NA>	<NA>		
35	<NA>		<NA>	<NA>		
47	<NA>		<NA>	<NA>		
	FanOfStarTrek	Gender	Age	HouseholdIncome	Education	
2	Yes	Male	18-29	\$0 - \$24,999	Bachelor degree	
11	<NA>	<NA>	<NA>	<NA>	<NA>	
12	<NA>	<NA>	<NA>	<NA>	<NA>	
26	Yes	Male	18-29	\$0 - \$24,999	High school degree	
35	Yes	Male	30-44	\$50,000 - \$99,999	Graduate degree	

47	No	Male 18-29	\$0 - \$24,999 High school degree
	Location		
2	West South Central		
11	<NA>		
12	<NA>		
26	East South Central		
35	East South Central		
47	Pacific		

```
> #View(look)
```

Component1 appears to picks off people who are not fans or have never seen the movies or otherwise provided trash data (lots of missing values, contradictions, etc). Lets subset the data according to this and rerun the analysis, computing principal components on this subset of data.

```
> subset = starwars.x[pca$x[,1]<4,]
> pca = prcomp(subset)
> plot(pca$x[,1:2])
```

Let's take a look at the biplot to see if we can find anything to note about the survey responses.

```
> biplot(pca, xlim = c(-.1,.15),ylim=c(-.12,.1),cex = c(.8, 1.5))
```

